

Calibrating the Instrument Before the Experiment: Two Oracle Failures and a Misestimated Variance Floor in a Procedurally Generated Agent Benchmark

rabidlego25

github.com/rabidlego25/agent_playground

30 August 2026

Abstract

We report the first two days of a long-running programme on agent configuration, whose motivating question is whether deliberation among several agents outperforms a single agent at matched token budgets. Before running that experiment we built a procedurally generated task suite with programmatic oracles and ran six calibration probes on it. The probes did not measure agent performance; they measured the apparatus, and they found it defective twice. First, the answer oracle compared a model’s final line verbatim against the gold string, so a model that narrated its conclusion scored zero on items it had answered correctly: one 8B model scored 2/30 on single-hop lookups it in fact solved. Second, after that repair, the task’s nominal difficulty parameter turned out to be decorative — the gold answer was the unique root of the generated graph in 50/50 instances at every depth, and a heuristic that ignores depth entirely scored 100%. Third, we find that variance across task *samples* (0.163) exceeds run-to-run variance on a fixed sample (0.050) by a factor of three, which constrains how any later arm comparison may be run. We argue that all three are instances of one failure mode — plausible, well-formed, confidently wrong measurement — and that the defence is storing complete traces rather than verdicts.

1 Introduction

Agent benchmarks report task completion. A benchmark for *configurations* — one agent versus k , communicating versus independent — must additionally survive comparisons at matched cost, which places far heavier demands on the measuring apparatus than a leaderboard does. An effect of five percentage points is a plausible size for a deliberation gain; it is also a plausible size for noise. Distinguishing them requires knowing the noise first.

This report covers the construction and calibration of that apparatus, undertaken 29–30 August 2026, and deliberately reports no result about any agent configuration. The planned experiment (“001”) compares five arms including a token-matched k -independent-samples control, on the grounds that “communication helps” is supported only if a communicating arm beats a non-communicating arm drawing the same number of samples. None of those arms has been run.

2 Methods

Tasks. Three seeded generator families emit instances together with a programmatic oracle: an assignment puzzle verified by brute force to admit exactly one solution, a code task scored against unseen test cases, and `multi_hop`, a reporting-chain lookup with a `depth` parameter. Procedural generation is intended to buy contamination resistance and an unlimited supply of instances at no token cost.

Traces. Every run is recorded as an episode of steps in append-only JSONL, with schema version, git commit, per-step token counts and latency, and the model’s complete response. A single-shot task is an episode of length one, so multi-step task families require no migration. The design intent is replay over re-run.

Backends. A provider-agnostic adapter exposes local models via `ollama` (`qwen2.5 7B`, `llama3.1 8B`) behind one interface. All results below are from those two models at 4-bit quantisation on an M1 Pro.

Statistics. Accuracies are reported with Wilson score intervals at 95%, since the normal approximation misbehaves near 0 and 1 at $n \approx 50$. Format compliance is reported as a separate column rather than folded into accuracy, for reasons that Section 3.1 makes clear.

3 Results

3.1 The oracle measured instruction-following

The first depth-curve run scored `llama3.1` at 2/30 on depth-1 lookups — a single hop. The traces showed the model answering correctly and being marked wrong: “*So, the person 1 level above Talos is Altair*” was scored incorrect because the oracle compared the final line verbatim to the gold name. The comparison model, `qwen2.5`, happened to emit bare names and scored normally.

Left alone this would have produced a six-point table with tight confidence intervals supporting an entirely fictional claim about relative reasoning ability. Nothing was malformed; no exception was raised; the numbers were plausible, because an 8B model being worse is a believable outcome. The repair was a delimited answer field (`ANSWER: <x>`) with strict parsing, a lenient fallback, and compliance recorded per response. The cost was a 15-fold increase in wall time, because a prompt that asks for a delimited answer invites the model to reason first.

Condition	<i>n</i>	acc	95% CI
<i>Distractor load (d=3)</i>			
0 distractors	50	0.96	[0.87, 0.99]
3	50	0.96	[0.87, 0.99]
6	50	0.88	[0.76, 0.94]
10	50	0.56	[0.42, 0.69]
14	50	0.56	[0.42, 0.69]
<i>Fact order</i>			
as generated	60	0.72	[0.59, 0.81]
shuffled	60	0.77	[0.65, 0.86]
<i>Entity names</i>			
real names	60	0.77	[0.65, 0.86]
nonsense tokens	60	0.72	[0.59, 0.81]

Table 1: Calibration probes, `qwen2.5-7B`. Format compliance was ≥ 0.97 in every cell and no cell recorded a backend error.

3.2 The difficulty knob was decorative

After the repair, the depth curve was still incoherent: `llama3.1` accuracy *rose* with depth (0.56 at $d=1$ to 0.84 at $d=6$) and `qwen2.5` peaked at $d=5$. Inspection of the generator showed why. Distractor edges were attached beneath existing nodes, so the chain top was always the unique root of the whole graph, and the gold answer was that root in 50/50 instances at every depth. A heuristic that walks upward until it cannot — never counting hops, never reading `depth` — scored 50/50 at $d=1$, $d=3$ and $d=6$. The parameter changed prompt length and nothing else.

The repair extends the chain $a \geq 1$ levels above the gold answer, making the answer an interior node. Re-testing over 200 instances per depth, the shortcut now scores 0/200 at every depth while the gold answer is verified to sit exactly `depth` hops above the query node in 200/200.

3.3 Calibration probes

Table 1 gives the probes that survive both repairs. Distractor count is a genuine difficulty knob. Neither shuffling the order of facts nor replacing entity names with nonsense tokens moves accuracy beyond the noise floor, which is the evidence that procedural generation is buying the contamination resistance it was adopted for.

3.4 Task-set variance dominates run-to-run variance

A dedicated probe re-ran an identical set of 60 tasks four times across two temperatures. The largest gap between cells was 0.050, which we initially adopted as the minimum detectable effect.

That number is wrong for the use it was intended for. The same nominal condition — `qwen2.5`, $d=3$, six distractors — recurs in five separate probes that differ only in which block of seeds they drew. Across those, accuracy ranges from 0.72 to 0.88, a spread of 0.163. Re-running fixed tasks captures sampling noise but not task-set noise, and the latter is roughly three times larger.

The consequence is a hard design constraint rather than a caveat: arms of a configuration experiment must be evaluated on *identical task instances*. Assigning each arm its own seed block would make a ten-point difference uninterpretable.

4 Discussion

The three defects share a structure. In each case the output was well formed, internally consistent, and of a magnitude a reader would accept. None produced an error or a warning. The first was caught only because 2/30 on a single-hop lookup is implausible on its face; the second only because an accuracy curve that rises with difficulty is implausible on its face; the third only because the same condition was measured five times by accident. Each detection depended on a redundancy that was not designed in.

Two practices follow. Store complete responses rather than verdicts, since every one of these diagnoses required reading text that a verdict-only log would have discarded. And measure the apparatus with quantities whose expected shape is known independently — monotonicity in a difficulty parameter, a floor at chance, agreement between nominally identical conditions — since a probe is useful precisely when it can fail visibly.

We note a bias in our own reporting: a calibration probe that produces an impressive number is likelier to have been misdesigned than to have discovered something, and the probes above are reported for what they constrain rather than for what they show.

5 Availability

Generators, trace schema, probes, and the raw JSONL traces for every run described here — including the two defective runs, retained under `results/confounded/` and `results/preshortcutfix/` — are at github.com/rabidlego25/agent.playground. Findings that motivated design changes are recorded as dated notes in `notes/`.